

P11 — Mini evaluador de expresiones algebraicas

Fundamentos de intérpretes o evaluador de expresiones

Integrantes

Código	Apellidos y nombres	Perfil en el proyecto

Descripción

Construir un evaluador de expresiones algebraicas aritméticas en C que soporte operadores básicos, funciones trigonométricas, logarítmicas, valor absoluto, máximo y mínimo, con una interfaz de consola interactiva similar al evaluador de GeoGebra.

Funcionalidades clave

- Lexer y parser recursivo descendente: tokenización de números, operadores (+, -, *, /, ^), paréntesis y nombres de función
- Funciones trigonométricas: sin(), cos(), tan(), asin(), acos(), atan(), atan2(); soporte de grados y radianes configurable
- Funciones logarítmicas y exponenciales: log(), log2(), log10(), exp(), sqrt(), pow(); constantes predefinidas pi y e
- Funciones de rango: abs() para valor absoluto, max(a,b) y min(a,b) para múltiples argumentos; operador módulo para enteros y reales
- Asignación de variables con letras (a = 3.5 + pi) y reutilización en expresiones posteriores; tabla de símbolos implementada con arreglo de structs
- Manejo de errores descriptivos: división por cero, argumento fuera de dominio (log de negativo, asin fuera de [-1,1]), paréntesis sin cerrar y función desconocida
- Interfaz REPL de consola (Read-Eval-Print Loop): historial de expresiones en memoria, comando list para ver variables definidas y comando quit para salir

Entregable 1 — Análisis y Diseño

Requerimientos funcionales y no funcionales del evaluador. Gramática formal (BNF/EBNF) del lenguaje de expresiones soportado. Diagrama de flujo del pipeline lexer→parser→evaluador con ejemplos de tokenización paso a paso. Diseño de la tabla de funciones implementada con arreglo de punteros a función (callbacks) y la tabla de símbolos para variables. Casos de prueba que cubran operadores básicos, precedencia, anidamiento de funciones y todos los errores de dominio.

Entregable 2 — Implementación

Código fuente en C organizado en módulos: lexer.c, parser.c, eval.c, functions.c, symtable.c, main.c con sus respectivos encabezados (.h). Makefile con compilación en una sola orden y linkeo a -lm. Documento de arquitectura explicando el diseño del AST, la tabla de funciones implementada con punteros a función, la tabla de símbolos y la estrategia de manejo de errores; incluir una transcripción de 10 expresiones de ejemplo evaluadas en la consola, comparando los resultados con los de GeoGebra.

Herramientas sugeridas

- GCC / Clang con -Wall -Wextra -lm (linkeo a la biblioteca matemática)
- Valgrind memcheck (nodos del AST con malloc/free) y AddressSanitizer
- Unity Test Framework para pruebas unitarias del lexer, parser y funciones matemáticas
- GDB con VS Code para depurar el recorrido del AST; gcov + lcov para cobertura de pruebas
- GitHub con Makefile y CI (compilación + tests automáticos en cada push); Doxygen para documentar la API de cada módulo